

Technologie multimedialne

Lista 7

Celem tej listy jest:

1. postprodukcja materiału wideo
2. transkodowanie materiału wideo
3. wykorzystanie [WebCodecs API](#) do analizy i ustawiania parametrów kompresji treści multimedialnych.

Lista na ocenę.

Z1 — postprodukcja filmu w programie DaVinci Resolve

Dokonaj postprodukcji materiału wideo (A/W). Pliki wejściowe znajdują się w podkatalogu projektu o nazwie `pliki we do Z1`. Przykładowy efekt końcowy zamieszczono w podkatalogu projektu o nazwie `pliki wy do Z1`.

Pliki wejściowe znajdują się w podkatalogu projektu o nazwie `pliki we do Z1`. Przykładowy efekt końcowy zamieszczono w podkatalogu projektu o nazwie `pliki wy do Z1`.

File Name

Location

Render ☒ Single clip ☐ Individual clips

Video Audio File

☒ Export Video

Format

Codec

Encoder

☐ Network Optimization

Resolution

☐ Use vertical resolution

Frame rate

30 frames per second

☐ Chapters from Markers

Encoding Profile

Key Frames ☒ Automatic ☐ Every frames

☒ Frame reordering

Preset

Rate Control

Quality



↑ Efekty przejścia między klatkami (stopniowe pojawianie się i znikanie obrazu).

← Przykładowe ustawienia parametrów wyjściowych: MP4/H.265, 2160p, 30 kl./s.

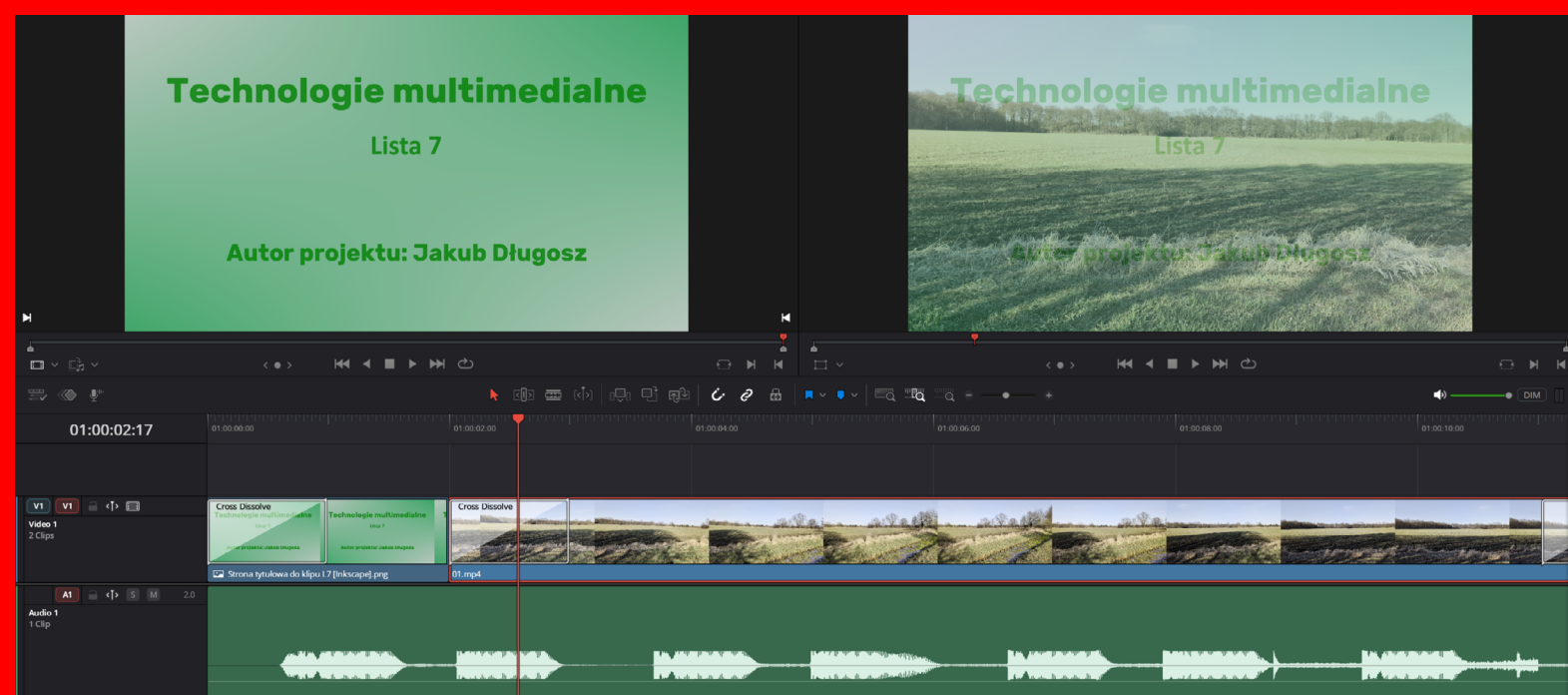
Kluczowe aspekty zadania:

1. Wyjściowy materiał wideo ma mieć następujące kontenery oraz kodeki:
 - a. MP4/H.264
 - b. MP4/H.265
 - c. webM.

Ponadto rozdzielczość wyjściowych plików wideo powinna wynosić 4K UHD (3840 x 2160 px). Skanowanie powinno być progresywne (2160p).

W nazwie pliku wyjściowego należy umieścić informacje dotyczące rozdzielczości obrazu oraz użytego kontenera i kodeku. Przykładowa nazwa pliku: L7_klip_wy_mp4_h265_4K.mp4.

2. W wyjściowym materiale wideo, na jego początku przez 2s, ma być umieszczona wstawka zawierająca tekst opisujący projekt.
3. Na początku i na końcu wyjściowego materiału wideo oraz na końcu wstawki mają występować efekty przejścia między klatkami (stopniowe pojawianie się i znikanie obrazu).
4. Z wejściowego pliku wideo (wideo_we.mp4) należy usunąć ścieżkę dźwiękową. W pliku wyjściowym ma być odtworzony początkowy fragment ścieżki audio z pliku audio_we.mp3, który powinien trwać tyle ile trwa ścieżka wideo. Głośność ścieżki audio powinna być niska, ma to być dźwięk w tle. W przypadku dodania kolejnej ścieżki audio z nagraniem głosowym, głos powinien być słyszalny.
5. W wyjściowym materiale wideo wejściowy materiał wideo powinien zostać przycięty, tak aby po 2s wstawce z tekstem wideo było odtworzone na chwilę przed momentem w którym kamera zaczyna poruszać się w prawą stronę.
6. Do zapisania materiału wyjściowego do formatu webM (plik o nazwie L7_klip_wy_4K.webm) należy użyć jednego z programów wcześniej omawianego na zajęciach.



↑ Przykładowy wygląd osi czasu w Z1.

Z2 — Sterowanie parametrami kodowania i segmentacja z [mp4box.js](#)

Zaimplementować rozszerzoną wersję demo [mp4box.js/demo/index.html](#), która używa WebCodecs API VideoEncoder do enkodowania strumienia wideo (np. z kamery lub z kontenera <video>) oraz pozwala użytkownikowi interaktywnie zmieniać parametry enkodera (bitrate, framerate, keyframe interval, codec profile/level, rate control mode) oraz przekazuje zakodowane ramki do mp4box.js w celu tworzenia segmentów fMP4 o konfigurowalnej długości. Celem jest zbadanie wpływu parametrów enkodera na rozmiar segmentów, opóźnienie i jakość.

Należy oddać:

1. zmodyfikowane pliki index.html oraz JS/CSS,
2. plik README.md z instrukcją uruchomienia i testów,
3. skrypt do uruchomienia lokalnego serwera,
4. przykładowe nagrania testowe (lub instrukcje ich generowania przez ffmpeg),
5. krótki raport (1–2 strony) opisujący obserwacje: wpływ parametrów na rozmiar segmentów, opóźnienie i jakość.

Wymagania funkcjonalne

1. **Interfejs użytkownika** z następującymi kontrolkami:
 - o wybór kodeka (np. avc1, vp09, hev1 – jeśli dostępne),
 - o przepływność bitowa (bitrate, kbps),
 - o klatkaż (framerate, kl./s, fps),
 - o liczba klatek między klatkami kluczowymi (keyframe interval),
 - o czas trwania segmentu (segment duration, ms),
 - o tryb przepływności (rate control, CBR/VBR – jeśli przeglądarka/kodek wspiera).
2. **Źródło wideo:** opcja wyboru między kamerą (getUserMedia) a lokalnym plikiem z kontenera <video> (przykładowe pliki testowe).
3. **WebCodecs VideoEncoder:**
 - o konfiguracja i (re)konfiguracja na bieżąco w czasie działania programu, bez przeładowania strony,
 - o obsługa encoder.encode() z ustawieniem keyFrame gdy wymagane,
 - o zbieranie zakodowanych fragmentów (EncodedChunk) i przekazywanie ich do mp4box.js.
4. **mp4box.js:**
 - o tworzenie ścieżki (track) w mp4box.js z otrzymanych fragmentów (chunk),
 - o segmentacja fMP4 z konfigurowalnym parametrem segmentDuration,
 - o zapis/eksport segmentów (do pamięci) oraz możliwość pobrania pojedynczego segmentu.
5. **Obliczenia i wizualizacja:**
 - o wykresy/wyświetlanie: przepływność bitowa na segment, rozmiar segmentu, opóźnienie enkodera (czas od VideoFrame do EncodedChunk), liczba klatek

kluczowych.

6. **Skrypt testowy:** skrypt (np. `Node + http-server` lub `serve`) i instrukcja uruchomienia.

Z3 — Sterowanie parametrami kodowania i dekodowania z użyciem [JsWebM](#) i [WebCodecs API](#)

Należy zaprojektować i zaimplementować eksperymentalny odtwarzacz/edytor webowy, który:

- **enkoduje i dekoduje** wideo przy użyciu [WebCodecs API](#),
- używa [JsWebM](#) do zapisu/odczytu kontenera WebM (multipleksacja i demultipleksacja),
- pozwala interaktywnie **sterować parametrami kodowania** (bitrate, framerate, keyframe interval, codec, rate control) oraz **parametrami dekodowania** (strategia buforowania, drop frames, hardware/software),
- wykorzystuje materiał wideo przygotowany i wyeksportowany lokalnie z darmowej wersji **DaVinci Resolve** (lub innego programu do postprodukcji wideo) z Z1.

Należy oddać:

- Kod projektu: `index.html`, `app.js`, `styles.css` oraz ewentualne moduły pomocnicze (integrujące z JsWebM).
- plik `README.md` z instrukcją uruchomienia i testów
- przykładowe nagrania testowe (lub instrukcje ich generowania przez `ffmpeg`).
- skrypt do uruchomienia lokalnego serwera.
- Raport: 2–4 strony z wynikami eksperymentów, wykresami i wnioskami.
- Opcjonalnie – krótkie nagranie pokazujące działanie UI i porównania.

Wymagania funkcjonalne

1. Interfejs użytkownika

- o Kontrolki do ustawiania parametrów enkodera: **codec** (np. `avc1`, `vp09`, `hev1` – jeśli dostępne), **bitrate** (kbps), **framerate** (fps), **keyframe interval**, **rate control mode** (jeśli przeglądarka wspiera).
- o Kontrolki dekodera: tryb dekodowania (hardware/software – jeśli wykrywalne), limit bufora (ms), strategia (wł./wył.) opuszczania ramek (drop frames on/off).
- o Panel logów i wykresów: rozmiar wygenerowanych fragmentów (chunk), opóźnienie enkodera, liczba klatek kluczowych, przepustowość (throughput).

2. Źródła wideo

- o Możliwość wczytania lokalnego pliku wideo oraz opcjonalnie przechwycenie strumienia wideo z kamery (`getUserMedia`).

3. Enkodowanie ([WebCodecs VideoEncoder](#))

- o Pobieranie ramek z kontenera `<video>` lub z `MediaStreamTrackProcessor` → `VideoFrame`.
- o Konfiguracja `VideoEncoder` zgodnie z ustawieniami UI; możliwość rekonfiguracji na bieżąco w czasie działania programu – restart enkodera z nowymi parametrami.
- o Zbieranie `EncodedChunk` i zapisywanie ich do bufora, a następnie przekazywanie do `jswebm` w celu utworzenia pliku WebM lub strumienia.

4. Zapis/odczyt z `jswebm`

- o Użycie `jswebm` do **multipleksacji** zakodowanych fragmentów (`chunk`) do kontenera WebM i do **demultipleksacji** WebM w celu odtworzenia.

- o Implementacja trybu „nagraj i odtwarzaj”: nagranie kilku sekund do WebM, zapis w pamięci (Blob), następnie odtworzenie przez demultiplekser → dekoder WebCodecs `VideoDecoder` lub bezpośrednio w kontenerze `<video>` (porównaj obie te opcje).

5. Dekodowanie (WebCodecs `VideoDecoder`)

- o Odtwarzanie zakodowanych fragmentów (chunk) przez `VideoDecoder.decode()` i renderowanie ramek w `HTMLCanvasElement` lub `VideoFrame` → `HTMLVideoElement` (jeśli możliwe).
- o Implementacja strategii dekodowania: bufor minimalny/maksymalny, drop frames on/off, funkcja zapasowa (fallback) powrotu do natywnego odtwarzacza.

6. Metryki i porównania jakości

- o Zbieranie metryk: rozmiar pliku/segmentu, przepływność bitowa (bitrate), opóźnienie enkodera (czas od `VideoFrame` do `EncodedChunk`), opóźnienie dekodera (czas pomiędzy `Encoded/Decoded Chunk` a wyświetleniem).
- o Eksperymenty przedstawione w raporcie: porównanie wyników dla 3 zestawów parametrów (np. niska/średnia/wysoka przepływność bitowa).

7. Powtarzalny scenariusz testowy

- o Skrypt lub instrukcja jak uruchomić testy lokalnie (np. utworzyć materiały W lub A/W o czasie trwania 10s, odtworzyć, zmierzyć podane metryki) umożliwiające ich uruchomienie na innym sprzęcie niż autora projektu.